Project 3 Companion Notes: Thought Cascade

Reference notes for `p03_Thought_Cascade.ipynb` (no separate report required)

ECE 4424 / CS 4824
Prof. Ming Jin (100 pts, 10%)

Project 3 places a quantized 1.7B language model (`Qwen3-1.7B`) inside a two-round code-repair loop and asks a concrete empirical question: when does test-runner feedback help a small model fix defective code, and when does it fail? Credit is based on meeting the function contracts, keeping the experimental comparison clean, and analyzing your saved runs honestly. Low model accuracy, unchanged starter code, or `0/3` success rates receive full credit when analyzed accurately; do not re-run seeds to look for a particular result.

1. Suggested workflow

Phase	Environment	Tasks	Completion check
1. Implement & test	CPU only (0 model calls)	Run Setup. Implement the six `TODO` functions (three in Q1, one each in Q4, Q5, Q6) and run the deterministic scripted checks before loading the model.	`check_q1`, `check_components`, and `check_loop` print `passed`.
2. Run experiments	Colab T4 GPU (or CUDA / MPS, ~10–15 min)	Select a T4 GPU, run Setup, and execute Q1 through Q6 once in order with seeds `11, 22, 33` (at most 57 model calls: 6 in Q1, 3 in Q3, up to 48 in Q6).	Tables, plots, paired checks, and `results/student_run.json` are saved.
3. Write & export	No GPU needed (0 model calls)	Write Q2 (200–350 words), Q3 (100–200 words), and Q6 (250–400 words) in the notebook Markdown cells. Save `.ipynb`, run `export_submission()`, and print HTML to PDF in Chrome.	`LastName_FirstName_Project3.pdf` shows all 6 functions, 3 answers, tables, and plots.

2. Tests, metrics, and failure modes (Q1–Q3)

- **Independent oracle and half-open intervals (Q1).** Do not test binary search with another binary search. Write a simple left-to-right scan in `leftmost_index_or_minus_one` and use it to label both explicit edge cases and random tests. In the starter's half-open interval `[left, right)`, `left` is included and `right` is excluded (`left == right` is empty). Check whether each branch update preserves that convention without skipping a valid index, and whether matching `target` returns too early when duplicates appear to the left.
- **Case pass rate vs. task success rate (Q1–Q2).** Passing 90 of 100 test cases on a single generated patch means `pass_rate = 0.90` and `all_passed = False` (`success_rate = 0.0` for that attempt). Those 100 cases evaluate one model attempt on 100 inputs, not 100 separate model calls.
- **Measurement change vs. behavior change (Q1–Q2).** Re-scoring the saved Q1 patches on `STRICT` after `MINIMAL` changes the test suite, not the generated code. Check `code_changed` as well: a candidate can repeat the starter unchanged (`False`) or edit the code (`True`) while introducing a new bug.

Failure type (Q2)	Meaning	Saved observation fields
Syntax error	Module failed to parse or compile; no test cases ran.	`compilation == 'error'`, `status == 'syntax_error'`, details in `error`.
Runtime exception	Module compiled and the suite ran (`status == 'tested'`), but one or more test inputs raised an exception. (Timeouts or blocked imports set `status` to `'timeout'` or `'rejected'`.)	`compilation == 'success'`, `runtime_errors > 0`, and `Error:` entries in `failures`.
Wrong answer	Function returned normally without raising, but produced the wrong output or mutated its input list.	`wrong_answers > 0` (or `input_mutations > 0`); compare `Expected` and `Actual` in `failures`.

3. Controller and paired experiment (Q4–Q6)

- **Current candidate vs. best candidate (Q6).** Later edits can degrade an earlier partial fix, so `react_loop` tracks two candidates. First evaluate the starter code on `dev_tests`; if it already passes all development tests, return immediately with zero steps and zero model calls. Otherwise, in round `i` (`0, ..., max_rounds - 1`):
  1. Build context from `current_code`; pass `current_observation` only when `feedback=True and i > 0`.
  2. Call `thought_fn` with `seed + 2*i` (`max_new_tokens=128`) and `action_fn` with `seed + 2*i + 1` (`max_new_tokens=512`).
  3. Check `validate_action(action)` before accessing `action['params']['code']`. If invalid, keep the previous `current_code` and record the required `invalid_action` observation; if valid, run `observe(task, code, dev_tests)` and update `current_code` and `current_observation` even if the score drops.
  4. Update `best_code` and `best_observation` only when `pass_rate` strictly exceeds the previous best (ties keep the earlier candidate). Stop early if `all_passed` is true on `dev_tests`.
- **Development vs. held-out split.** Development tests play the role of a validation set (used for Round 2 feedback, stopping, and selecting `best_code`). The 100 held-out cases are evaluated once after `react_loop` finishes and never enter prompts or selection.
- **Paired Round 1 check.** In Round 1 (`i = 0`), neither condition receives test observations and both use the same seeds. For each seed, the Round 1 prompt, plan, and patch should match across `feedback=False` and `feedback=True`, isolating the effect of Round 2 test feedback.
- **What differs between the two Q6 tasks.** For binary search, the specification already states the full contract, so failing tests provide concrete counterexamples of an already stated rule. For `decode_status`, the prompt lists codes (`R2`, `H7`, `M4`) and labels without their correspondence, so failing development tests supply missing external information. When comparing conditions across 3 seeds, note sample-size limits (including floor or ceiling effects) and compare token counts and runtime alongside accuracy.

4. Function contract notes

Function	Required behavior	Common mistake to avoid
`build_strict_tests`	Place at least 10 distinct explicit cases first (empty, singleton hit and miss, unique first and last index, absent below/between/above, negatives, duplicates), followed by `n_random` cases from a local `random.Random(seed)`.	Calling global `random.seed()`, deduplicating across the combined list (changing total length), or omitting singleton miss/hit.
`measure_single_shot_success`	Do not mutate `records`; keep syntax/runtime failures in the denominator `len(records)`; raise `ValueError` on empty input.	Filtering out non-compiling attempts or modifying dictionaries in place.
`generate_thought` & `generate_action`	Include `context` unchanged (and `thought` in Q5). Keep prompts task-independent. Return the generation record in Q4, and `(action, record)` in Q5.	Hard-coding task names, binary-search hints, or status mappings into the prompt string.
`react_loop`	Validate actions before reading `params['code']`; return `selected_code` from the best development candidate rather than the last round.	Passing observations in Round 1 or when `feedback=False`, or overwriting `best_code` on ties.

5. Submission

Keep all edits inside notebook cells (rerunning Setup overwrites files in `.thought_cascade/`). Before exporting, confirm that `check_q1`, `check_components`, and `check_loop` pass, Q6's paired first-round checks are `True`, and your written answers for Q2, Q3, and Q6 cite your saved tables and logs. Save or download the completed `.ipynb`, run `export_submission()`, open the generated `*_submission.html` in Chrome, and use **Print > Save as PDF** (with headers and footers unchecked) to submit a single file named `LastName_FirstName_Project3.pdf`. If no free GPU is available after passing Stage 1 checks, ask course staff for recorded logs and label them as recorded.